

EDU GENIE

Google Gemini Powered Learning Assistant

PROJECT REPORT

TEAM LEADER:
SRI DHARSHINI (2522K0778)

TEAM MEMBERS
NIRMAL (2522K0775)
SRIRAM (2522K0779)
ASWIN KUMAR (2522K0768)

DEPARTMENT / INSTITUTION:

BSC CS

ACADEMIC YEAR: 2026–2027

DECLARATION

I hereby declare that the project entitled “EDUGENIE – Google Gemini Powered Learning Assistant” is my original work carried out by me under the guidance of DEIVIKA during the academic year 2026–2027.

I further declare that this project has not been submitted previously, either wholly or partially, to any other institution or university for the award of any degree, diploma, or certificate.

TEAM LEADER SRI DHARSHINI

TEAM MEMBER NIRMAL, SRIRAM, ASWIN KUMAR

CERTIFICATE

This is to certify that SRI DHARSHINI, NIRMAL, SRIRAM & ASWIN KUMAR, a student of SRI RAMALINGA SOWDAMBIGAI COLLEGE OF SCIENCE AND COMMERCE, has successfully completed the project entitled "EDUGENIE – Google Gemini Powered Learning Assistant" under my guidance and supervision during the academic year 2026–2027.

The project work submitted is an original work carried out by the student in partial fulfillment of the requirements for the completion of the course.

PROJECT GUIDE:	DEIVIKA
HEAD OF THE DEPARTMENT:	VISALAKSHI
DATE:	30-09-2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who supported and guided me in completing the project entitled “EDUGENIE – Google Gemini Powered Learning Assistant.”

I sincerely thank the management and Principal of DR. P. BALASUBRAMANIAN for providing me with the necessary facilities and support to complete this project successfully.

I express my heartfelt thanks to the Head of the Department, VISALAKSHI, for their valuable guidance and encouragement throughout the project.

I am especially grateful to my project guide, DEIVIKA, for their continuous guidance, valuable suggestions, and support during the development of this project.

I also thank all my teachers, friends, and family members for their encouragement and assistance. Finally, I thank everyone who contributed directly or indirectly to the successful completion of this project.

ABSTRACT

EDUGENIE – Google Gemini Powered Learning Assistant is an AI-powered educational application designed to provide students with an interactive and personalized learning experience. The system integrates the capabilities of Google Gemini to assist learners in understanding academic concepts, answering questions, generating explanations, summarizing study materials, and providing relevant examples.

The main objective of EDUGENIE is to make learning more accessible, engaging, and efficient by providing students with an intelligent virtual learning assistant. Through natural-language interaction, students can ask questions and receive easy-to-understand responses based on their learning requirements.

The system helps students clarify doubts, revise topics, explore new concepts, and obtain learning support at their own pace. It can also reduce the time required to search through multiple learning resources by providing AI-assisted explanations in a single platform.

EDUGENIE demonstrates the practical application of generative artificial intelligence in the field of education. By combining modern AI technology with an easy-to-use learning interface, the system aims to support students in their academic journey and promote a more interactive and personalized approach to education.

TABLE OF CONTENTS

Chapter / Section	Section Title	Page Number
1.	INTRODUCTION	1
2.	PROBLEM STATEMENT	2
3.	OBJECTIVES	3
4.	EXISTING AND PROPOSED SYSTEM	4
5.	SYSTEM REQUIREMENTS	5
6.	SYSTEM ARCHITECTURE	6
7.	MODULE DESIGN	7
8.	DATABASE AND DATA DESIGN	8
9.	API DESIGN	9
10.	CODING AND IMPLEMENTATION	10
11.	FRONTEND DESIGN	12
12.	PROMPT ENGINEERING	13
13.	ALGORITHMS AND PSEUDOCODE	14
14.	TESTING AND VALIDATION	15
15.	SECURITY AND RESPONSIBLE AI	16
16.	DEPLOYMENT	17
17.	RESULTS AND DISCUSSION	18
18.	ADVANTAGES AND LIMITATIONS	19
19.	FUTURE ENHANCEMENTS	20
20.	CONCLUSION	21
21	REFERENCES	22
22	TEAM MEMBER	23

1. INTRODUCTION

EDUGENIE – Google Gemini Powered Learning Assistant is an AI-based educational application developed to provide students with intelligent and interactive learning support. The system uses Google Gemini's generative AI capabilities to understand user queries and provide relevant explanations, answers, summaries, examples, and study assistance. EDUGENIE aims to make learning more convenient and personalized by allowing students to interact with an AI assistant using natural language. The system can support students in understanding difficult topics, clearing doubts, revising lessons, and improving their overall learning experience.

1.1 Project Scope

- Interactive educational question answering
- Simple explanations for difficult concepts
- Automatic multiple-choice quiz generation
- Condensing long passages into revision notes
- Topic-based beginner-to-advanced learning paths
- REST API integration between browser and backend
- Modular Python implementation for maintainability

1.2 Target Users

- School and college students
- Self-learners
- Teachers preparing quick revision activities
- Developers learning how to integrate generative AI into web applications

2. PROBLEM STATEMENT

Students often face difficulties in understanding complex academic concepts, finding suitable learning resources, and getting immediate answers to their questions. Traditional learning methods may not always provide personalized assistance according to each student's learning speed and requirements. Students may also spend considerable time searching through different websites and study materials. EDUGENIE addresses these challenges by providing an AI-powered learning assistant that can respond to student queries, explain concepts in a simple manner, summarize information, and provide examples through an interactive platform.

2.1 Proposed Problem Solution

Problem	EduGenie Response
Complex academic text	Simplification module
Doubt while studying	Q&A module
Need for practice	Quiz module
Long material	Summary module
Unclear study sequence	Learning Path module

3. OBJECTIVES

The primary objective of EDUGENIE is to develop an intelligent learning assistant that provides quick and personalized educational support. The system aims to help students clarify their doubts, understand academic concepts, summarize study materials, and generate useful examples. Another objective is to provide an easy-to-use interface through which students can communicate naturally with the AI assistant. The project also aims to demonstrate how generative AI can be effectively integrated into education while promoting responsible and secure use of AI technology.

- Develop a modular AI-powered learning assistant.
- Provide a clean browser interface for common study operations.
- Expose separate FastAPI endpoints for each learning function.
- Use structured prompts to make AI outputs useful for learners.
- Validate quiz output so it can be safely rendered by the frontend.
- Provide clear errors when AI responses or requests are invalid.
- Design the project so additional modules can be added later.

3.1 Measurable Functional Goals

Goal	Expected Behaviour
Q&A	Returns a concise answer to a learner question
Explain	Converts a topic into beginner-friendly explanation
Quiz	Produces MCQs with options and answers
Summarize	Returns key ideas in shorter form
Recommend	Produces a staged learning path

4. EXISTING AND PROPOSED SYSTEM

In the existing system, students generally depend on textbooks, teachers, search engines, educational websites, and online videos to obtain learning support. These methods can require considerable time and may not always provide answers according to the student's specific requirements. The proposed EDUGENIE system provides an AI-powered alternative that allows students to interact with a learning assistant through natural-language questions. By integrating Google Gemini, the proposed system can generate contextual responses, explanations, summaries, and learning assistance in a centralized platform.

4.1 Existing Learning Workflow

In a conventional workflow, students search separately for explanations, summaries, questions and tutorials. The quality and format of results may vary, and the learner has to organize the material manually.

4.2 Proposed EduGenie Workflow

EduGenie provides a single task selector and input area. The frontend sends a POST request to the appropriate FastAPI endpoint. The backend validates the request, builds a task-specific prompt, invokes the configured model, normalizes the response and sends the result back to the browser.

High-Level Learning Workflow

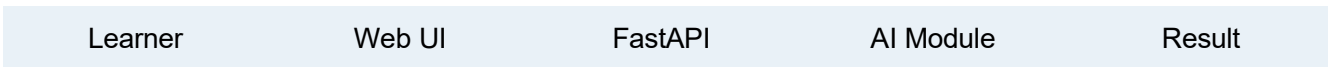


Figure: High-Level Learning Workflow

4.3 Comparative View

Aspect	Conventional Workflow	EduGenie
Interface	Multiple sites	Single interface
Task switching	Manual	Task selector
Quiz format	Find/create separately	Generated by module
Learning path	Manual research	AI recommendation
Architecture	External services	Modular API design

5. SYSTEM REQUIREMENTS

The system requirements describe the hardware and software resources required to develop and operate EDUGENIE. A computer or mobile device with a stable internet connection is required for accessing the application and communicating with the AI service. The software environment may include a modern web browser, a suitable programming language and development framework, a database system where required, and access to the Google Gemini API. The system should also have sufficient processing capability, memory, and storage for application development, testing, and deployment.

5.1 Hardware

- Modern laptop or desktop computer
- At least 4 GB RAM for basic development; more is useful for local models
- Internet connection for cloud AI requests
- Browser such as Chrome, Edge or Firefox

5.2 Software

- Python 3.10+
- FastAPI
- Uvicorn
- Jinja2
- HTML5 and CSS3
- Configured Google Gemini API access
- Optional local transformer model for explanation

5.3 Installation Commands Environment Setup

The EDUGENIE – Google Gemini Powered Learning Assistant requires a suitable development environment with Node.js, Visual Studio Code, Git, a modern web browser, and access to the Google Gemini API. First, create the project using `mkdir edugenie` and `cd edugenie`, then initialize the frontend using `npm create vite@latest frontend -- --template react` followed by `npm install`. For the backend, create a separate directory using `mkdir backend`, initialize it with `npm init -y`, and install the required packages using `npm install express cors dotenv`. The Gemini API key should be stored securely in a `.env` file using `GEMINI_API_KEY=your_api_key_here` and should not be exposed in the source code. The application can be started locally using `npm run dev` for the frontend and `node server.js` for the backend. Before deployment, the project can be prepared using `npm run build`. This environment setup provides the necessary foundation for developing, testing, configuring, and deploying the EDUGENIE learning assistant.

6. SYSTEM ARCHITECTURE

The architecture of EDUGENIE consists of several interconnected components that work together to provide an AI-based learning experience. The user interacts with the frontend interface by entering a question or learning request. The frontend sends the request to the backend, where it is processed and prepared for communication with the Gemini API. Google Gemini generates an appropriate response based on the submitted prompt, and the backend sends the response back to the frontend for display. Database components may be used to store relevant user information, conversation data, preferences, or application records.

Layered Architecture

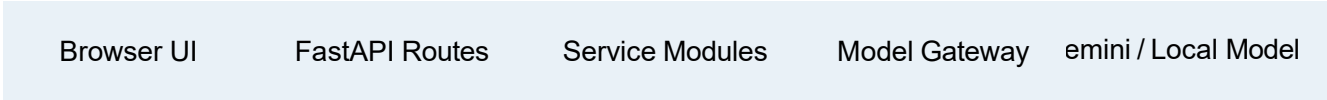


Figure: Layered Architecture

6.1 Request Flow

- User selects a task and enters text.
- Frontend sends JSON or form data to the relevant endpoint.
- FastAPI validates the request.
- The service module creates a structured prompt.
- The AI provider generates a response.
- The service validates and normalizes the response.
- FastAPI returns JSON or text.
- Frontend displays the result.

6.2 Failure Flow Error Handling Flow

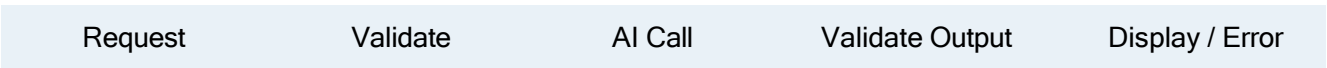


Figure: Error Handling Flow

7. MODULE DESIGN

EDUGENIE can be divided into multiple modules to simplify development and maintenance. The user interface module allows students to interact with the application. The authentication module can manage user registration and login where required. The AI interaction module handles communication with Google Gemini and processes user queries. The learning assistance module provides features such as explanations, summaries, question answering, and study support. A database module can manage application data, while an administration module can be included for managing users and system information.

7.1 Question Answering Module

The Q&A module accepts a natural-language question and requests a direct educational answer. Prompt instructions can request clarity, concise language and appropriate examples.

7.2 Explanation Module

The explanation module focuses on reducing complexity. A topic can be converted into a definition, core idea, example and short recap. The source project associates this function with a lightweight LaMini-Flan-T5 model.

7.3 Quiz Module

The quiz module generates multiple-choice questions from supplied text or a topic. A structured JSON format is preferred because the frontend can render question cards programmatically.

7.4 Summary Module

The summary module removes repetition while preserving important concepts. It can be configured for a target length or bullet-point output.

7.5 Learning Path Module

The recommendation module produces a progression from beginner concepts to more advanced topics, with suggested resources and a logical study order.

8. DATABASE AND DATA DESIGN

The database and data design define how information used by EDUGENIE is organized and managed. Depending on the implementation, the system may store user profiles, login information, conversation history, questions, AI-generated responses, learning preferences, and other application-related data. Appropriate database structures and relationships can be designed to avoid unnecessary duplication and improve data retrieval. Sensitive information should be handled carefully, and only the data required for the application's functionality should be stored.

Entity	Important Fields	Purpose
User	user_id, name, created_at	Learner profile
Session	session_id, user_id, started_at	Interaction grouping
Activity	activity_id, type, input, output	Learning history
QuizAttempt	quiz_id, score, timestamp	Assessment tracking
Progress	user_id, topic, level, streak	Learning analytics

8.1 Example SQL Proposed SQLite Schema

```
CREATE TABLE users (  
    user_id INTEGER PRIMARY KEY, name  
    TEXT NOT NULL,  
    created_at TEXT NOT NULL  
);  
  
CREATE TABLE activities (  
    activity_id INTEGER  
    PRIMARY KEY, user_id INTEGER,  
    activity_type TEXT NOT NULL, input_text  
    TEXT NOT NULL, output_text TEXT,  
    created_at TEXT NOT NULL  
);
```

9. API DESIGN

The API design defines how different components of EDUGENIE communicate with each other and with external services. The application can use backend API endpoints to receive user queries, process requests, communicate securely with the Google Gemini service, and return generated responses to the frontend. APIs may also be used for authentication, user management, conversation storage, and other application functions. Proper request validation, error handling, authentication, and secure management of API credentials are important parts of the API design.

Endpoint	Method	Input	Output
/qa	POST	question	answer
/explain	POST	topic	explanation
/quiz	POST	passage	questions
/summarize	POST	text	summary
/learn/recommendations	POST	topic	learning path

9.1 Example Request Q&A JSON

```
{
  "question": "Explain photosynthesis in simple words."
}
```

9.2 Example Response Q&A Response

```
{
  "answer": "Photosynthesis is the process by which green plants..."
}
```

10. CODING AND IMPLEMENTATION

The coding and implementation phase involves converting the proposed design into a working application. The frontend is developed to provide an interactive interface, while the backend handles application logic, user requests, data processing, and communication with the Gemini API. Appropriate programming practices such as modular coding, input validation, error handling, and reusable functions are followed. The implementation is tested during development to identify and correct functional errors and to ensure that the different components work together properly.

10.1 FastAPI Application main.py

```
from fastapi import FastAPI from pydantic
import BaseModel

app = FastAPI(title="EduGenie API") class

TextRequest(BaseModel):
    text: str

class QuestionRequest(BaseModel): question:
    str

@app.get("/") def
home():
    return {"message": "EduGenie is running"}

@app.post("/qa")
def qa(req: QuestionRequest):
    return {"answer": generate_answer(req.question)}

@app.post("/summarize")
def summarize(req: TextRequest):
    return {"summary": generate_summary(req.text)}
```

10.2 AI Service Layer ai_service.py

```
import os

def generate_answer(question: str) -> str: prompt = f"""
    You are an educational assistant.
    Answer the following question clearly for a student:
    {question} """
    return call_model(prompt)

def generate_summary(text: str) -> str: prompt = f"""
    Summarize the following educational text. Keep the main
    facts and use simple language. TEXT:
    {text} """
    return call_model(prompt)
```

10.3 Quiz JSON Normalization quiz_module.py

```
import json

def clean_json_block(raw: str): raw =
    raw.strip()
    if raw.startswith("```"):
        raw = raw.replace("```json", "").replace("```", "").strip() return json.loads(raw)

def validate_quiz(items):
    if not isinstance(items, list):
        raise ValueError("Quiz must be a list") for item in items:
        if len(item.get("options", [])) != 4:
            raise ValueError("Each question needs four options") if "answer" not in
            item:
                raise ValueError("Missing correct answer") return items
```

11.FRONTEND DESIGN

The frontend of EDUGENIE is designed to provide a simple, responsive, and user-friendly learning environment. It can include features such as a login page, dashboard, chat interface, input box, response area, navigation options, and learning-related controls. The interface should allow students to submit questions easily and read AI-generated responses clearly. Responsive design principles can be used so that the application remains usable across computers, tablets, and mobile devices.

11.1 HTML Interface templates/index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>EduGenie</title>
  <link rel="stylesheet" href="/static/style.css">
</head>
<body>
  <main class="container">
    <h1>EduGenie</h1>
    <p>AI-powered learning assistant</p>

    <select id="task">
      <option value="qa">Ask a Question</option>
      <option value="explain">Explain a Topic</option>
      <option value="quiz">Generate Quiz</option>
      <option value="summarize">Summarize</option>
      <option value="recommend">Learning Path</option>
    </select>

    <textarea id="input" placeholder="Enter your topic or text"></textarea>
    <button onclick="runTask()">Submit</button>
    <section id="result"></section>
  </main>
</body>
</html>
```

11.2 CSS static/style.css

```
body {
  font-family: Arial, sans-serif; margin: 0;
  background: #f4f7fb;
}
.container {
  max-width: 800px; margin:
  40px auto; padding: 24px;
  background: white; border-
  radius: 12px;
}
textarea { width:
  100%;
  min-height: 180px;
  margin: 12px 0;
}
button {
  padding: 10px 18px;
  cursor: pointer;
}
```

12. PROMPT ENGINEERING

Prompt engineering plays an important role in obtaining useful and consistent responses from Google Gemini. EDUGENIE uses carefully designed prompts to guide the AI in providing educational responses appropriate for students. Prompts can specify the required format, explanation level, language, tone, and context of the response. For example, the system can instruct Gemini to explain a complex topic using simple language and suitable examples. Well-designed prompts help improve the relevance, clarity, and usefulness of generated responses.

12.1 Q&A Prompt Template Prompt

ROLE: You are a patient educational tutor. TASK: Answer the student's question.

RULES:

- Use simple language.
- Explain the core idea first.
- Add one short example when useful.
- Do not invent facts when the input is uncertain. QUESTION: {question}

12.2 Quiz Prompt Template Prompt

Create 3 multiple-choice questions from the passage. Return ONLY valid JSON:

```
[
  {
    "question": "...",
    "options": ["A", "B", "C", "D"],
    "answer": "A"
  }
]
```

Passage: {text}

12.3 Learning Path Prompt Prompt

Create a learning path for: {topic} Include:

1. Beginner foundations
2. Intermediate concepts
3. Advanced concepts
4. Practice activities
5. Suggested resource types Use a clear numbered sequence.

13.ALGORITHMS AND PSEUDOCODE

The algorithms used in EDUGENIE define the logical steps followed when processing user requests. When a student enters a question, the system validates the input, prepares the appropriate prompt, sends the request to the Gemini API, receives the generated response, and displays it to the student. If an error occurs, the system provides an appropriate message instead of displaying incomplete information. Pseudocode can be used to represent these steps in a simple and programming-independent format before implementation.

13.1 Q&A Algorithm Pseudocode

```
START
Read question
Validate question is not empty Build educational prompt
Send prompt to AI service Receive response
If response is valid: return answer
Else:
    return error message
END
```

13.2 Quiz Algorithm Pseudocode

```
START
Read passage
Build JSON-output prompt Request questions from AI
Clean Markdown wrappers if present Parse JSON
Validate number of options Validate answer field Return quiz
END
```

13.3 Summary Algorithm Pseudocode

```
START
Read source text
Check minimum input length Build summary prompt Generate response
Check response exists Return concise summary END
```

13.4 Learning Path Algorithm Pseudocode

```
START
Read topic
Ask AI for staged progression
Separate beginner, intermediate and advanced sections Attach
practice/resource suggestions
Return formatted learning path
END
```

14. TESTING AND VALIDATION

Testing and validation are performed to ensure that EDUGENIE functions correctly and provides reliable results. Functional testing verifies features such as user input, authentication, API communication, response generation, and navigation. Integration testing checks whether the frontend, backend, database, and Gemini API work together correctly. Usability testing evaluates whether students can easily understand and use the interface. Error and edge-case testing is also performed to verify how the system behaves when invalid input, network problems, or API failures occur.

Test ID	Scenario	Expected Result
T01	GET /	Service status message
T02	Empty Q&A	Validation/error response
T03	Valid question	Answer returned
T04	Quiz generation	Questions contain four options
T05	Malformed quiz JSON	Readable error; no crash
T06	Long summary text	Shorter summary returned
T07	Learning path	Ordered progression returned
T08	Unknown endpoint	HTTP 404
T09	Browser submit	Result appears in UI
T10	API unavailable	Frontend shows useful error

14.1 Test Execution Checklist

- Start the Uvicorn server.
- Open the browser interface.
- Run one request for each task.
- Inspect the raw API response.
- Check frontend formatting.
- Repeat with empty and unusually long inputs.
- Record errors and improve validation.

15. SECURITY AND RESPONSIBLE AI

Security and responsible AI are important considerations when developing an AI-powered educational application. API keys and other sensitive credentials should not be exposed in the frontend or public source code. User data should be protected through appropriate authentication, authorization, secure communication, and controlled data storage. Since AI-generated responses may sometimes contain inaccurate or incomplete information, EDUGENIE should encourage users to verify important academic information with trusted sources. The system should also avoid generating harmful, discriminatory, or inappropriate content and should promote responsible use of AI.

- Never place a private API key directly in frontend JavaScript.
- Use environment variables for secrets.
- Validate request size and content.
- Handle provider failures without exposing sensitive configuration.
- Avoid storing learner data unless there is a clear purpose and appropriate protection.
- Encourage users to verify important academic facts with trusted materials.
- Use structured output validation before rendering AI-generated data.

15.1 Environment Configuration env example

```
GEMINI_API_KEY=your_key_here  
MODEL_NAME=your_configured_model
```

15.2 Responsible Educational Use

EduGenie is best treated as a learning companion rather than an unquestionable authority. Students should use generated explanations to support understanding and should consult textbooks, teachers or authoritative sources when accuracy is important.

16. DEPLOYMENT

Deployment is the process of making EDUGENIE available for actual users. The frontend, backend, database, and required API services can be deployed using suitable hosting platforms depending on the project architecture. Environment variables can be used to securely configure API keys and other sensitive settings. Before deployment, the application should be tested in a production-like environment to verify performance, connectivity, security, and functionality. Proper monitoring and maintenance can help ensure reliable operation after deployment

16.1 Local Run Command

```
uvicorn main:app --reload # Open:  
http://127.0.0.1:8000
```

16.2 Suggested Deployment Structure Deployment



Figure: Deployment

16.3 Production Considerations

- Use HTTPS.
- Store secrets in managed environment configuration.
- Disable verbose debug output.
- Add request limits and logging.
- Use a process manager for reliable service operation.
- Monitor provider errors and response latency.

17. RESULTS AND DISCUSSION

The implementation of EDUGENIE demonstrates how generative AI can be used to provide interactive learning assistance. The system can accept natural-language questions and generate explanations, summaries, examples, and other educational responses. The conversational approach allows students to ask follow-up questions and explore topics according to their learning requirements. The results demonstrate the potential of integrating Google Gemini into an educational application, while also highlighting the importance of verifying AI-generated information.

Feature	User Input	System Output
Q&A	Question	Educational answer
Explain	Topic	Simplified explanation
Quiz	Passage/topic	MCQs with options
Summary	Long text	Concise revision text
Learning Path	Topic	Staged learning plan

17.1 Example User Journey

A learner enters “Database normalization” and selects Explain. After reading the result, the learner can switch to Quiz and submit the same topic for practice. The learner can then request a Learning Path to identify prerequisite and advanced topics. This creates a simple study loop: understand → practice → organize next steps.

18. ADVANTAGES AND LIMITATIONS

EDUGENIE provides several advantages, including quick access to learning assistance, natural-language interaction, personalized explanations, easy access to summaries, and support for self-paced learning. It can reduce the time students spend searching for basic explanations and provide assistance whenever the service is available. However, the system also has limitations. AI-generated responses may occasionally contain incorrect or incomplete information, and the quality of responses can depend on the clarity of user prompts and the capabilities of the AI model. Internet connectivity and API availability may also affect system operation.

18.1 Advantages

- Single interface for several study tasks.
- Modular backend is easier to maintain.
- Structured quiz output supports automated rendering.
- Cloud AI can provide broad language understanding.
- Lightweight frontend is easy to run and modify.
- Future modules can be added without redesigning the entire application.

18.2 Limitations

- Generated answers can contain inaccuracies.
- Cloud AI requires network access and service availability.
- API usage may introduce cost or rate limits.
- Local models may require additional memory and setup.
- Personalization is limited without user history and progress data.
- Generated learning resources should be checked before use.

19.FUTURE ENHANCEMENTS

EDUGENIE can be enhanced in the future by adding more advanced educational features. Possible enhancements include personalized learning plans, quizzes and assessments, progress tracking, voice-based interaction, multilingual support, document and PDF analysis, automatic study schedules, flashcard generation, and integration with educational platforms. The system could also provide teacher dashboards and learning analytics to help monitor student progress. Additional safety and verification mechanisms could be introduced to improve the reliability of AI-generated educational content.

- Voice-based question answering for hands-free study.
- Multilingual interface and multilingual explanations.
- Mobile application for Android and iOS.
- Progress dashboard with topic mastery indicators.
- Gamification using badges, streaks and challenges.
- Teacher/parent dashboards with appropriate privacy controls.
- Moodle or Google Classroom integration.
- Image and PDF input for snapshot-based doubt solving.
- Adaptive learning paths using quiz performance.
- Offline or cached revision mode where technically feasible.

19.1 Proposed Future Architecture Future EduGenie Ecosystem

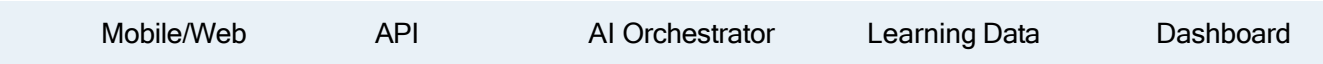


Figure: Future EduGenie Ecosystem

20. CONCLUSION

EduGenie demonstrates how generative AI can be organized into practical educational workflows rather than being used only as an open-ended chatbot. The project combines a FastAPI backend with a simple browser interface and task-specific modules for Q&A, explanation, quiz generation, summarization and learning recommendations.

The expanded implementation in this report adds clearer API contracts, validation patterns, sample Python code, frontend code, prompt templates, pseudocode, testing cases and deployment considerations. These additions provide a foundation for further development while retaining the core architecture and terminology of the supplied EduGenie project.

The project also highlights an important principle for educational AI: generated content should assist learning, while students and educators remain responsible for verifying important information and applying academic judgment.

21. REFERENCES

The references section contains the sources used during the design, development, implementation, and documentation of EDUGENIE. These may include official Google Gemini and Google AI documentation, programming-language documentation, framework documentation, database documentation, research papers related to generative AI and education, and other reliable educational technology resources. All external information, libraries, APIs, and technical concepts used in the project should be properly referenced according to the required academic citation format.

- Supplied project document: “EduGenie: Google Gemini Powered Learning Assistant.”
- Google AI for Developers documentation (for Gemini API concepts).
- FastAPI documentation (for REST API development).
- Python documentation (for Python language and environment setup).
- Uvicorn documentation (for ASGI application serving).
- Jinja2 documentation (for server-side HTML templating).

TEAM DETAILS

Team Leader	SRI DHARSHINI	2522K0778
Member	NIRMAL	2522K0775
Member	SRIRAM	2522K0779
Member	ASWIN KUMAR	2522K0768